

OpenCL/OpenGL Interoperation + MeshFree FEM

Julian Panetta

5 March 2013



NEW YORK UNIVERSITY



- Cross-platform API for visualization on the GPU



- Cross-platform API for visualization on the GPU
- Includes shader languages (GLSL)
 - Run code once per vertex, per rasterized pixel, etc.



- Cross-platform API for visualization on the GPU
- Includes shader languages (GLSL)
 - Run code once per vertex, per rasterized pixel, etc.
 - Add great flexibility on top of the traditional “fixed function pipeline”



- Cross-platform API for visualization on the GPU
- Includes shader languages (GLSL)
 - Run code once per vertex, per rasterized pixel, etc.
 - Add great flexibility on top of the traditional “fixed function pipeline”
 - Can be tricked into doing some general purpose GPU computing, but...



- Cross-platform API for visualization on the GPU
- Includes shader languages (GLSL)
 - Run code once per vertex, per rasterized pixel, etc.
 - Add great flexibility on top of the traditional “fixed function pipeline”
 - Can be tricked into doing some general purpose GPU computing, but...
 - Computations like this tend to be hacks



- “Open Computing Language:”
Simple C-like language for SIMD-type parallel programming



- “Open Computing Language:”
Simple C-like language for SIMD-type parallel programming
- Generates vectorized code to run on:
 - Multiple CPU cores



- “Open Computing Language:”
Simple C-like language for SIMD-type parallel programming
- Generates vectorized code to run on:
 - Multiple CPU cores
 - GPUs



- “Open Computing Language:”
Simple C-like language for SIMD-type parallel programming
- Generates vectorized code to run on:
 - Multiple CPU cores
 - GPUs
 - Cell architectures, etc.



- “Open Computing Language:”
Simple C-like language for SIMD-type parallel programming
- Generates vectorized code to run on:
 - Multiple CPU cores
 - GPUs
 - Cell architectures, etc.
- Much better suited for computation than GLSL shaders



- “Open Computing Language:”
Simple C-like language for SIMD-type parallel programming
- Generates vectorized code to run on:
 - Multiple CPU cores
 - GPUs
 - Cell architectures, etc.
- Much better suited for computation than GLSL shaders
- Often better-suited for vectorized CPU code (SSE, AVX) than C and/or intrinsics



- Typical setting: you want to run a single operation on thousands of different inputs



- Typical setting: you want to run a single operation on thousands of different inputs
- Write a *kernel* program implementing the operation



- Typical setting: you want to run a single operation on thousands of different inputs
- Write a *kernel* program implementing the operation
- Launch thousands of “concurrent” instances of the program, one per input



- Typical setting: you want to run a single operation on thousands of different inputs
- Write a *kernel* program implementing the operation
- Launch thousands of “concurrent” instances of the program, one per input
- Examples
 - 2D image convolution (OpenCL kernel implements convolution kernel)



- Typical setting: you want to run a single operation on thousands of different inputs
- Write a *kernel* program implementing the operation
- Launch thousands of “concurrent” instances of the program, one per input
- Examples
 - 2D image convolution (OpenCL kernel implements convolution kernel)
 - Eigenvalues of hundreds of 3x3 matrices



- Typical setting: you want to run a single operation on thousands of different inputs
- Write a *kernel* program implementing the operation
- Launch thousands of “concurrent” instances of the program, one per input
- Examples
 - 2D image convolution (OpenCL kernel implements convolution kernel)
 - Eigenvalues of hundreds of 3x3 matrices
 - Many more: see your local HPC class

- Both OpenGL and OpenCL have “contexts”

- Both OpenGL and OpenCL have “contexts”
- OpenGL contexts manage:
 - OpenGL state (transform matrices, rendering styles, etc.)

- Both OpenGL and OpenCL have “contexts”
- OpenGL contexts manage:
 - OpenGL state (transform matrices, rendering styles, etc.)
 - GLSL Programs

- Both OpenGL and OpenCL have “contexts”
- OpenGL contexts manage:
 - OpenGL state (transform matrices, rendering styles, etc.)
 - GLSL Programs
 - Memory buffers (vertex buffers, frame buffers, textures)

- Both OpenGL and OpenCL have “contexts”
- OpenGL contexts manage:
 - OpenGL state (transform matrices, rendering styles, etc.)
 - GLSL Programs
 - Memory buffers (vertex buffers, frame buffers, textures)
- OpenCL contexts manage:
 - Kernels

- Both OpenGL and OpenCL have “contexts”
- OpenGL contexts manage:
 - OpenGL state (transform matrices, rendering styles, etc.)
 - GLSL Programs
 - Memory buffers (vertex buffers, frame buffers, textures)
- OpenCL contexts manage:
 - Kernels
 - Memory buffers

Typical (GPU) OpenCL Program Flow

- Create OpenCL context on a device (GPU)

Typical (GPU) OpenCL Program Flow

- Create OpenCL context on a device (GPU)
- Compile OpenCL Kernels

Typical (GPU) OpenCL Program Flow

- Create OpenCL context on a device (GPU)
- Compile OpenCL Kernels
- Allocate memory buffers on the GPU

Typical (GPU) OpenCL Program Flow

- Create OpenCL context on a device (GPU)
- Compile OpenCL Kernels
- Allocate memory buffers on the GPU
- Copy input data from CPU to GPU's input buffers

Typical (GPU) OpenCL Program Flow

- Create OpenCL context on a device (GPU)
- Compile OpenCL Kernels
- Allocate memory buffers on the GPU
- Copy input data from CPU to GPU's input buffers
- Run the kernel

Typical (GPU) OpenCL Program Flow

- Create OpenCL context on a device (GPU)
- Compile OpenCL Kernels
- Allocate memory buffers on the GPU
- Copy input data from CPU to GPU's input buffers
- Run the kernel
- Copy output data from the GPU's buffers back to the CPU

Typical (GPU) OpenCL Program Flow

- Create OpenCL context on a device (GPU)
- Compile OpenCL Kernels
- Allocate memory buffers on the GPU
- Copy input data from CPU to GPU's input buffers
- Run the kernel
- Copy output data from the GPU's buffers back to the CPU
- Post-process (visualize?)

Typical (GPU) OpenCL Program Flow

- Create OpenCL context on a device (GPU)
 - Compile OpenCL Kernels
 - Allocate memory buffers on the GPU
 - Copy input data from CPU to GPU's input buffers
 - Run the kernel
 - Copy output data from the GPU's buffers back to the CPU
 - Post-process (visualize?)
-
- Transferring data to/from device adds significant overhead!
 - Note: when the device is a CPU, all copies can be eliminated...



- Ideally we'd leave the computation on the GPU for visualization.



- Ideally we'd leave the computation on the GPU for visualization.
- We need OpenGL and OpenCL contexts to reference the same memory.



- Ideally we'd leave the computation on the GPU for visualization.
- We need OpenGL and OpenCL contexts to reference the same memory.
- This memory could be VBOs (vertex buffers), textures on the OpenGL side.



- Ideally we'd leave the computation on the GPU for visualization.
- We need OpenGL and OpenCL contexts to reference the same memory.
- This memory could be VBOs (vertex buffers), textures on the OpenGL side.
- We'll show how to have OpenCL write to a texture that OpenGL draws to the screen.

- Create an OpenCL context on the same GPU, telling it to share with the active OpenGL context.

Sharing GPU Data: Overview

- Create an OpenCL context on the same GPU, telling it to share with the active OpenGL context.
- Allocate the texture with OpenGL calls (glGenTextures, etc.)

Sharing GPU Data: Overview

- Create an OpenCL context on the same GPU, telling it to share with the active OpenGL context.
- Allocate the texture with OpenGL calls (`glGenTextures`, etc.)
- Create OpenCL “buffers” that just point to the OpenGL texture (`clCreateFromGLTexture`)

Sharing GPU Data: Overview

- Create an OpenCL context on the same GPU, telling it to share with the active OpenGL context.
- Allocate the texture with OpenGL calls (glGenTextures, etc.)
- Create OpenCL “buffers” that just point to the OpenGL texture (clCreateFromGLTexture)
- Request OpenCL ownership of texture

Sharing GPU Data: Overview

- Create an OpenCL context on the same GPU, telling it to share with the active OpenGL context.
- Allocate the texture with OpenGL calls (glGenTextures, etc.)
- Create OpenCL “buffers” that just point to the OpenGL texture (clCreateFromGLTexture)
- Request OpenCL ownership of texture
- Run kernel, writing to the texture

Sharing GPU Data: Overview

- Create an OpenCL context on the same GPU, telling it to share with the active OpenGL context.
- Allocate the texture with OpenGL calls (glGenTextures, etc.)
- Create OpenCL “buffers” that just point to the OpenGL texture (clCreateFromGLTexture)
- Request OpenCL ownership of texture
- Run kernel, writing to the texture
- Release ownership of texture

- Create an OpenCL context on the same GPU, telling it to share with the active OpenGL context.
- Allocate the texture with OpenGL calls (glGenTextures, etc.)
- Create OpenCL “buffers” that just point to the OpenGL texture (clCreateFromGLTexture)
- Request OpenCL ownership of texture
- Run kernel, writing to the texture
- Release ownership of texture
- Render with OpenGL

- Create an OpenCL context on the same GPU, telling it to share with active OpenGL context.

- Create an OpenCL context on the same GPU, telling it to share with active OpenGL context.

This bit is platform dependent (Apple, WGL, GLX)

- Create an OpenCL context on the same GPU, telling it to share with active OpenGL context.

This bit is platform dependent (Apple, WGL, GLX)

```
CGLContextObj kCGLContext = CGLGetCurrentContext();
CGLShareGroupObj kCGLShareGroup = CGLGetShareGroup(kCGLContext);

cl_context_properties props[] = {
    CL_CONTEXT_PROPERTY_USE_CGL_SHAREGROUP_APPLE,
    (cl_context_properties) kCGLShareGroup,
    0
};

cl_int status;
context = clCreateContext(props, 0, 0, NULL, 0, &status);
```

Sharing GPU Data: Buffer Allocation

- Allocate the texture with OpenGL calls (`glGenTextures`, etc.)
- Create OpenCL “buffers” that just point to the OpenGL texture (`clCreateFromGLTexture`)

- Allocate the texture with OpenGL calls (glGenTextures, etc.)
- Create OpenCL “buffers” that just point to the OpenGL texture (clCreateFromGLTexture)

```
GLuint tex;
glGenTextures(1, &tex);
glBindTexture(GL_TEXTURE_2D, tex);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
             GL_UNSIGNED_BYTE, NULL);
d_img = clCreateFromGLTexture(context, CL_MEM_READ_WRITE, GL_TEXTURE_2D,
                              0, tex, &err);
```

Sharing GPU Data: Visualization

- Request OpenCL ownership of texture
- Run kernel, writing to the texture
- Release ownership of texture
- Render with OpenGL

Sharing GPU Data: Visualization

- Request OpenCL ownership of texture
- Run kernel, writing to the texture
- Release ownership of texture
- Render with OpenGL

```
glFinish();  
clEnqueueAcquireGLObjects(queue, 1, &d_img, 0, NULL, NULL);  
clEnqueueNDRangeKernel(queue, kernel, 2, NULL, gdim, ldim,  
                        0, NULL, NULL));  
clEnqueueReleaseGLObjects(queue, 1, &d_img, 0, NULL, NULL);  
clFinish(queue);
```

- Request OpenCL ownership of texture
- Run kernel, writing to the texture
- Release ownership of texture
- Render with OpenGL

```
glFinish();  
clEnqueueAcquireGLObjects(queue, 1, &d_img, 0, NULL, NULL);  
clEnqueueNDRangeKernel(queue, kernel, 2, NULL, gdim, ldim,  
                        0, NULL, NULL));  
clEnqueueReleaseGLObjects(queue, 1, &d_img, 0, NULL, NULL);  
clFinish(queue);
```

- glFinish/clFinish ensure GL/CL commands each finish before the other is run

- Request OpenGL ownership of texture
- Run kernel, writing to the texture
- Release ownership of texture
- Render with OpenGL

```
glFinish();  
clEnqueueAcquireGLObjects(queue, 1, &d_img, 0, NULL, NULL);  
clEnqueueNDRangeKernel(queue, kernel, 2, NULL, gdim, ldim,  
                        0, NULL, NULL));  
clEnqueueReleaseGLObjects(queue, 1, &d_img, 0, NULL, NULL);  
clFinish(queue);
```

- glFinish/clFinish ensure GL/CL commands each finish before the other is run
- Benchmark warning: without glFinish, clEnqueueAcquireGLObjects takes non-negligible time

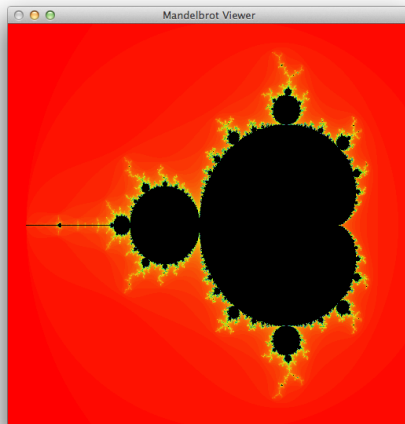
- The OpenCL buffer bound to the texture is of type `image2d_t`

Sharing GPU Data: The Kernel

- The OpenCL buffer bound to the texture is of type `image2d_t`
- must use `write_imagef` to write to the texture.

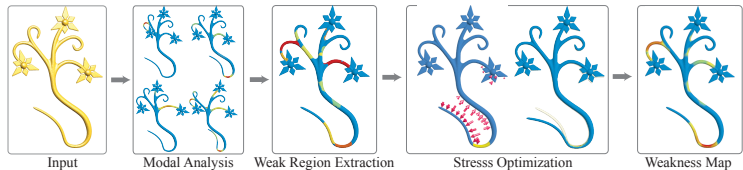
```
__kernel void mandelbrot(write_only image2d_t img, int w, int h,
                        float xMin, float xMax, float yMin, float yMax,
                        __global float *colorMap, unsigned int maxIters)
{
    ...
    write_imagef(img, (int2)(col, row),
                 (float4)(colorMap[colorOffset + 0],
                         colorMap[colorOffset + 1],
                         colorMap[colorOffset + 2], 1.0f));
    ...
}
```

Demo: OpenCL Mandelbrot Viewer

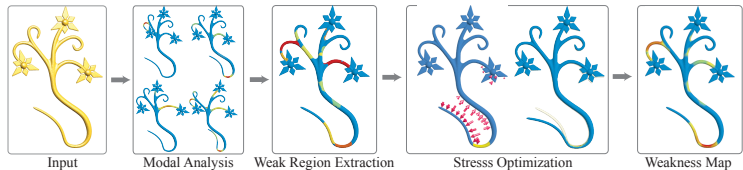


Demo

- James recently described our work on structural analysis.
- Pipeline:

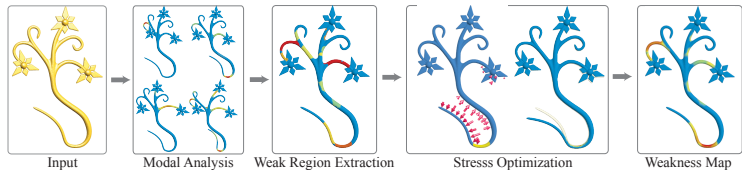


- James recently described our work on structural analysis.
- Pipeline:



- Missing initial stage: volume meshing!

- James recently described our work on structural analysis.
- Pipeline:



- Missing initial stage: volume meshing!
- There are several reasons to want to avoid meshing.

Mesh-free FEM: Why?

- Complicated geometry can be hard to mesh robustly.

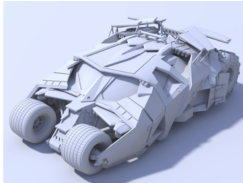
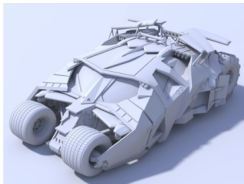


Image Source: Wikipedia

Mesh-free FEM: Why?

- Complicated geometry can be hard to mesh robustly.

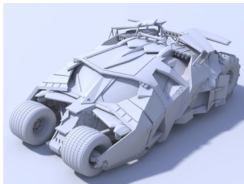


- Free ourselves from the surface resolution (we might want to coarsen some areas).

Image Source: Wikipedia

Mesh-free FEM: Why?

- Complicated geometry can be hard to mesh robustly.



- Free ourselves from the surface resolution (we might want to coarsen some areas).
- Sometimes you don't even have a mesh (CSG)

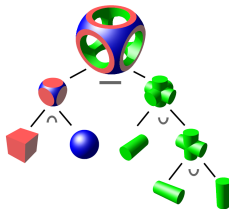


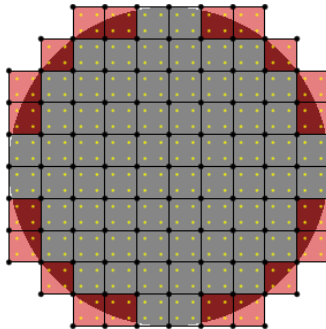
Image Source: Wikipedia

Mesh-free FEM: What?

- Use FEM discretization, but only require inside/outside geometry point queries

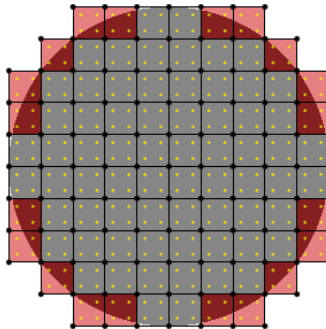
Mesh-free FEM: What?

- Use FEM discretization, but only require inside/outside geometry point queries
- There *is* still a mesh: a regular grid.

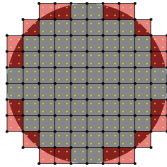


Mesh-free FEM: What?

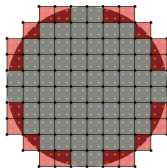
- Use FEM discretization, but only require inside/outside geometry point queries
- There *is* still a mesh: a regular grid.



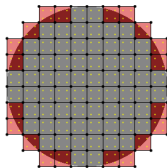
- Note: doesn't conform to the object's boundary.



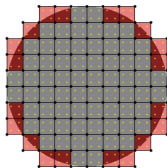
- Elements are grid cells overlapping the object.



- Elements are grid cells overlapping the object.
- Bilinear basis functions interpolate DoFs on the element corners.



- Elements are grid cells overlapping the object.
- Bilinear basis functions interpolate DoFs on the element corners.
- Strain is no longer piecewise constant! Integrals get trickier.



- Elements are grid cells overlapping the object.
- Bilinear basis functions interpolate DoFs on the element corners.
- Strain is no longer piecewise constant! Integrals get trickier.
- Integrands are not continuous on boundary cells (defined to be zero outside object) $\implies$ many quadrature points needed

- Visualization

- Visualization
 - FEM equations yield values on grid, not geometry vertices

- Visualization
 - FEM equations yield values on grid, not geometry vertices
 - Displacement fields must be used to deform the geometry somehow...

- Visualization
 - FEM equations yield values on grid, not geometry vertices
 - Displacement fields must be used to deform the geometry somehow...
 - Solution: Fancy GLSL shader warping texture coordinates

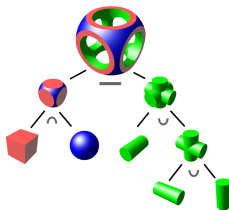
- Visualization
 - FEM equations yield values on grid, not geometry vertices
 - Displacement fields must be used to deform the geometry somehow...
 - Solution: Fancy GLSL shader warping texture coordinates
- Boundary cells without enough quadrature points... (see Demo)

- Visualization
 - FEM equations yield values on grid, not geometry vertices
 - Displacement fields must be used to deform the geometry somehow...
 - Solution: Fancy GLSL shader warping texture coordinates
- Boundary cells without enough quadrature points... (see Demo)
- Boundary conditions!
 - Boundaries don't have explicit representation, so applying Dirichlet/Neumann boundary conditions is tricky

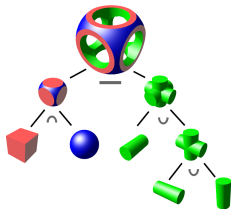
- Visualization
 - FEM equations yield values on grid, not geometry vertices
 - Displacement fields must be used to deform the geometry somehow...
 - Solution: Fancy GLSL shader warping texture coordinates
- Boundary cells without enough quadrature points... (see Demo)
- Boundary conditions!
 - Boundaries don't have explicit representation, so applying Dirichlet/Neumann boundary conditions is tricky
 - We have only Neumann conditions: traction applied to the surface

- Visualization
 - FEM equations yield values on grid, not geometry vertices
 - Displacement fields must be used to deform the geometry somehow...
 - Solution: Fancy GLSL shader warping texture coordinates
- Boundary cells without enough quadrature points... (see Demo)
- Boundary conditions!
 - Boundaries don't have explicit representation, so applying Dirichlet/Neumann boundary conditions is tricky
 - We have only Neumann conditions: traction applied to the surface
 - We plan to blur the surface forces to convert them into volume forces that can be integrated over the cells

- Apply structural analysis to CSG trees:

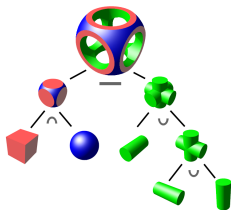


- Apply structural analysis to CSG trees:



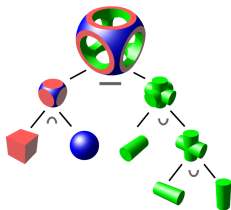
- Render geometry to texture using OpenCL
 - Note: can't use recursion!

- Apply structural analysis to CSG trees:



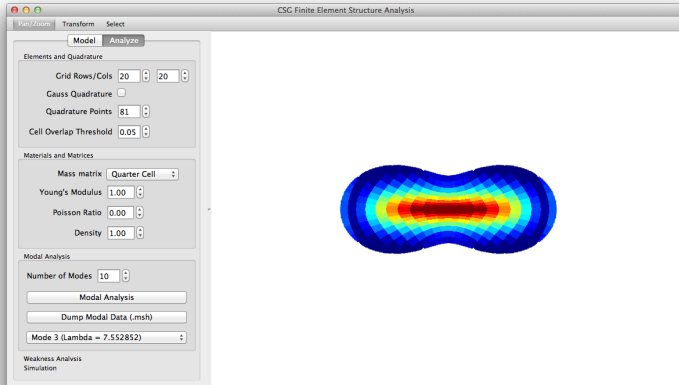
- Render geometry to texture using OpenCL
 - Note: can't use recursion!
- Visualize deformations using GLSL/OpenGL

- Apply structural analysis to CSG trees:



- Render geometry to texture using OpenCL
 - Note: can't use recursion!
- Visualize deformations using GLSL/OpenGL
- Currently have modal analysis working...

Demo: Mesh-free FEM



Demo